

SWEX Tower Anbindung

Diese Anleitung beschreibt den Standardweg, um ein Produktprojekt mit dem SWEX Tower zu verbinden:

- Stripe-Umsätze und Checkouts in den Tower spiegeln
- AI-/Support-Tickets in den Tower schreiben
- Customer, Sale und Support sauber mit `hubId`, `projectId` und `externalRef` zuordnen

Die Anleitung basiert auf der funktionierenden Anbindung aus `repzept-system-apotheken` und kann fuer andere Projekte wie das Digitale Wartezimmer wiederverwendet werden.

Zielbild

Ein Produktprojekt sendet zwei Arten von Daten an den SWEX Tower:

1 Stripe-/Checkout-/Billing-Daten

- Customer
- Sale / Order
- Payment Status

1 AI-/Support-Daten

- Support-Ticket
- Kundenbezug
- Projektbezug

SWEX Tower Endpunkte

Base URL:

`https://swex-control-tower.onrender.com`

Verwendete Endpunkte:

- Stripe-/Checkout-Ingest:

`POST /api/ingest/stripe-order`

- AI-/Support-Ingest:

`POST /api/ingest/ai-support-ticket`

Wichtig:

- Nicht mit alten oder internen Pfaden wie `/customers/upsert` arbeiten
- Für externe Produktsysteme immer die `/api/ingest/...`-Endpunkte verwenden

Tokens und Zuständigkeiten

Im SWEX Tower gibt es zwei relevante Ingest-Tokens:

- `STRIPE_INGEST_TOKEN`
- `SUPPORT_INGEST_TOKEN`

Wichtig:

- `POST /api/ingest/stripe-order` akzeptiert `STRIPE_INGEST_TOKEN` oder `SUPPORT_INGEST_TOKEN`
- `POST /api/ingest/ai-support-ticket` akzeptiert nur `SUPPORT_INGEST_TOKEN`

Das bedeutet:

- Stripe kann mit eigenem Stripe-Ingest-Token angebunden werden
- Support braucht immer einen gültigen Support-Ingest-Token

Benötigte Render-Variablen im Produktprojekt

Diese Variablen müssen im jeweiligen Produktservice auf Render gesetzt werden:

```
SWEX_TOWER_BASE_URL=https://swex-control-tower.onrender.com
SWEX_STRIPE_INGEST_PATH=/api/ingest/stripe-order
SWEX_SUPPORT_INGEST_PATH=/api/ingest/ai-support-ticket
SWEX_STRIPE_INGEST_TOKEN=<stripe-ingest-token>
SWEX_SUPPORT_INGEST_TOKEN=<support-ingest-token>
SWEX_HUB_ID=<hub-id>
SWEX_PROJECT_ID=<fallback-project-id>
```

Optional:

```
SWEX_DEFAULT_PROJECT_ID=<fallback-project-id>
```

Wenn das Produkt intern pro Kunde/Praxis/Apotheke eine eigene `projectId` speichert, sollte diese bevorzugt werden. `SWEX_PROJECT_ID` ist dann nur Fallback.

Benötigte Variablen im SWEX Tower

Im Service `swex-control-tower` auf Render:

```
SUPPORT_INGEST_TOKEN=< langer-zufallswert >
```

STRIPE_INGEST_TOKEN=<optional-separater-wert>

Wenn kein eigener Stripe-Token genutzt wird, kann technisch auch nur mit SUPPORT_INGEST_TOKEN gearbeitet werden. Sauberer ist aber eine Trennung.

Zu sendende Stripe-/Sale-Daten

Empfohlenes Payload fuer:

POST /api/ingest/stripe-order

```
{
  "hubId": "DEINE_HUB_ID",
  "projectId": "DEINE_PROJECT_ID",
  "customerName": "Max Mustermann",
  "customerFirstName": "Max",
  "customerLastName": "Mustermann",
  "customerEmail": "max@example.com",
  "customerPhone": "+49123456789",
  "company": "Muster GmbH",
  "customerAddress": "Musterstr. 1, 80331 Muenchen",
  "productName": "Produktname",
  "amountEur": 99,
  "currency": "EUR",
  "billingCycle": "monthly",
  "paymentStatus": "paid",
  "checkoutSessionId": "cs_test_123",
  "paymentReference": "stripe-session-or-payment-id",
  "externalRef": "eigene-interne-referenz"
}
```

Wichtige Felder:

- hubId: Hub im SWEX Tower
- projectId: Projekt im SWEX Tower
- externalRef: interne Referenz aus dem Produkt
- paymentStatus: bei erfolgreichem Event auf paid
- paymentReference: Stripe-Checkout-/Invoice-/Payment-Referenz

Zu sendende Support-Daten

Empfohlenes Payload fuer:

POST /api/ingest/ai-support-ticket

```
{
  "hubId": "DEINE_HUB_ID",
  "projectId": "DEINE_PROJECT_ID",
  "title": "Kunde braucht Hilfe",
  "description": "First-Level-KI konnte das Problem nicht loesen. Beschreibung...",
  "priority": "medium",
  "customerEmail": "kunde@example.com",
  "customerName": "Max Mustermann",
}
```

```
"sourceModel": "gpt-5-codex",  
"externalRef": "produkt-ticket-123",  
"language": "de"  
}
```

Stripe Webhook im Produktprojekt

Wenn Stripe-Events serverseitig verarbeitet werden, wird im Produktprojekt ein Webhook-Endpoint benoetigt.

Beispiel:

```
https://dein-produkt.example.com/api/stripe/webhook
```

Empfohlene Stripe-Events:

- `checkout.session.completed`
- `invoice.paid`
- `customer.subscription.created`
- `customer.subscription.updated`
- `payment_intent.succeeded`

Zusätzliche Render-/Produktvariablen fuer den Webhook:

```
STRIPE_SECRET_KEY=<stripe-secret-key>  
STRIPE_WEBHOOK_SECRET=<whsec_...>
```

Zuordnung im Produkt

Jedes Produkt braucht eine saubere interne Zuordnung von Stripe zu seinem eigenen Fachobjekt.

Bewaehrt hat sich:

- lokale Speicherung von `stripeCustomerId`
- lokale Speicherung von `stripeSubscriptionId`
- interne `externalRef`
- lokales Mapping auf Kunden-/Praxis-/Apothekenobjekt

Typische Quellen fuer das Matching:

- `metadata.entityType`
- `metadata.entityId`
- `stripeCustomerId`
- `stripeSubscriptionId`

- `customerEmail`

Idempotenz

Stripe-Events koennen mehrfach kommen. Deshalb muss das Produkt Duplikate verhindern.

Empfohlene Sicherungen:

- `stripeEventId` eindeutig speichern
- `externalRef` eindeutig machen
- falls vorhanden:
- `checkoutSessionId`
- `invoiceId`
- `paymentIntentId`

So wurde es im Rezept-Projekt verifiziert

1. Domain und Webhook

- Produktdomain auf Render eingerichtet
- Stripe Webhook auf den Produktendpoint gesetzt

2. Stripe-Ingest

Erfolgreich gegen den echten Tower getestet:

- Customer wurde angelegt
- Order/Sale wurde angelegt
- `paymentStatus = paid`

3. Support-Ingest

Erfolgreich gegen den echten Tower getestet:

- Support-Ticket wurde ueber `/api/ingest/ai-support-ticket` angelegt
- Antwort 201

Checkliste für neue Projekte

Im SWEX Tower

- 1 `SUPPORT_INGEST_TOKEN` setzen oder rotieren

- 2 optional STRIPE_INGEST_TOKEN setzen
- 3 hubId und projectId festlegen

Im Produktprojekt

- 1 SWEX_TOWER_BASE_URL setzen
 - 2 SWEX_STRIPE_INGEST_PATH setzen
 - 3 SWEX_SUPPORT_INGEST_PATH setzen
 - 4 SWEX_STRIPE_INGEST_TOKEN setzen
 - 5 SWEX_SUPPORT_INGEST_TOKEN setzen
 - 6 SWEX_HUB_ID setzen
 - 7 SWEX_PROJECT_ID oder projektbezogene projectId setzen
 - 8 STRIPE_SECRET_KEY setzen
 - 9 STRIPE_WEBHOOK_SECRET setzen
10. Stripe-Webhook-Endpunkt im Stripe-Dashboard eintragen

Funktionstest

- 1 echten oder kontrollierten Stripe-Event auslösen
- 2 prüfen, ob im Tower Customer + Sale auftauchen
- 3 Support-Test senden
- 4 prüfen, ob Ticket im Tower auftaucht

Typische Fehlerbilder

- 404 Not found
 - falscher Endpoint
 - meist alter Pfad statt /api/ingest/...
-
- 401 Invalid ingest token
 - falscher Token
 - Stripe-Token fuer Support verwendet
 - Support-Token im Tower nicht gesetzt
-
- 503 ... not configured
 - Token im Tower selbst nicht als Env gesetzt
-
- keine Zuordnung im Produkt
 - Stripe-Kunde kann keinem lokalen Objekt zugeordnet werden

- fehlende metadata, stripeCustomerId oder E-Mail-Zuordnung

Empfehlung für künftige Projekte

Fuer jedes neue Projekt immer gleich standardisieren:

- dieselbe Ingest-Architektur nutzen
- externalRef als interne Referenz konsequent pflegen
- Stripe- und Support-Token getrennt halten
- hubId und projectId frueh sauber definieren
- erst preview/Dry-Run, dann echter Write